

Graham Scan

1 Overview

Graham Scan

Graham Scan computes the convex hull by sorting points around a pivot and then using a stack to remove points that create invalid turns.

The algorithm has two main ideas:

- Sort points in angular order around a guaranteed hull point.
- Scan the sorted points and remove any point that creates a right turn or an unwanted collinear point.

Core Intuition

While walking around the outside boundary, the hull must keep turning in one consistent direction. In counterclockwise order, the valid turn is a left turn.

2 Input and Output

Input:

$$S = \{P_1, P_2, \dots, P_n\}.$$

Output:

$$H = (H_1, H_2, \dots, H_m), \quad m \leq n,$$

where H is the convex hull boundary in order.

3 Main Steps

1. Choose a pivot point P_0 .
2. Sort all other points by polar angle around P_0 .
3. If multiple points have the same angle, keep the farthest one when only corner vertices are required.
4. Push the first few sorted points onto a stack.
5. For each next point, use orientation to test the turn.
6. Pop stack points while the turn is invalid.
7. Return the stack as the hull.

4 Choosing the Pivot

Choose the point with the smallest y -coordinate. If there is a tie, choose the point with the smallest x -coordinate.

$$P_0 = \text{point with minimum } y, \text{ breaking ties by minimum } x.$$

This point is guaranteed to lie on the convex hull because no point is below it.

5 Polar-Angle Sorting

Sort all other points counterclockwise by their polar angle around P_0 .

Squared Distance

If two points have the same polar angle from P_0 , compare squared distances:

$$d^2(P_0, Q) = (q_x - p_x)^2 + (q_y - p_y)^2.$$

Keep the farther point and discard the nearer point if the hull output should contain only corner vertices.

Squared distance is enough because the square root does not change the ordering.

Boundary Convention

Keeping only the farthest point for each polar angle removes points that lie on the same ray from the pivot. This matches the convention that the convex hull output contains only corner vertices. If an implementation must return every boundary point, same-angle and collinear cases need a different tie-handling rule.

6 Stack Rule

Suppose the top two stack points are A and B , and the next scanned point is C . Compute:

$$\text{orient}(A, B, C).$$

For

$$A = (A_x, A_y), \quad B = (B_x, B_y), \quad C = (C_x, C_y),$$

the orientation value is

$$\text{orient}(A, B, C) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x).$$

Orientation value	Stack action
$\text{orient}(A, B, C) > 0$	Left turn. Keep B , then push C .
$\text{orient}(A, B, C) < 0$	Right turn. Pop B .
$\text{orient}(A, B, C) = 0$	Collinear. Usually keep the farther point and discard the nearer point.

Stack Invariant

After each scan step, the stack stores the current outer boundary candidate in correct order.

7 Pseudocode

Graham Scan

```
GrahamScan(points):
    remove duplicate points

    if number of points <= 2:
        return points

    p0 = point with minimum y-coordinate
        breaking ties by minimum x-coordinate

    sort remaining points by polar angle around p0

    for points with the same polar angle:
        keep only the farthest point from p0

    if number of remaining sorted points == 0:
        return [p0]

    if number of remaining sorted points == 1:
        return [p0, first sorted point]

    stack = empty stack
    push p0
    push first sorted point
    push second sorted point

    for each remaining point p:
        while stack has at least two points
            and orientation(next_to_top, top, p) <= 0:
                pop stack

        push p

    return stack
```

This version removes right turns and interior collinear points.

8 Correctness Proof

Let the points after pivot selection and polar-angle sorting be

$$P_0, P_1, P_2, \dots, P_m,$$

where P_0 is the pivot and $P_1, \dots, P_m$ are sorted counterclockwise by polar angle.

Loop Invariant

At the start of each iteration that processes P_i , the stack S , read from bottom to top, contains exactly the vertices of

$$\text{CH}(\{P_0, P_1, \dots, P_{i-1}\})$$

in counterclockwise order.

Initialization

Before the loop starts,

$$S = [P_0, P_1, P_2].$$

These are exactly the hull vertices of the first three processed points, so the invariant is true.

Maintenance

Assume the invariant is true before processing P_i . Let

$$A = \text{NextToTop}(S), \quad B = \text{Top}(S).$$

The algorithm tests the turn

$$\text{orient}(A, B, P_i).$$

If the turn is not left, then B lies inside the new boundary or on one of its edges, so B cannot remain a hull corner and is popped. The while-loop removes all such invalid top points. When a left turn is finally obtained, pushing P_i gives the hull of

$$\{P_0, P_1, \dots, P_i\}$$

in counterclockwise order. Thus the invariant is maintained.

Termination

When the loop ends, all points have been processed. By the invariant, S contains exactly the vertices of

$$\text{CH}(\{P_0, P_1, \dots, P_m\}).$$

Therefore Graham Scan returns the convex hull vertices in counterclockwise order.

Correctness Result

Graham Scan returns exactly the convex hull vertices in counterclockwise order.

9 Worked Example

Point Set

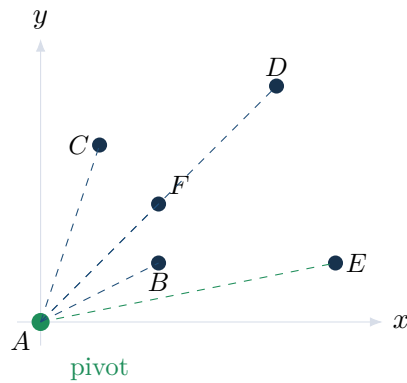
$$A = (0, 0), \quad B = (2, 1), \quad C = (1, 3),$$

$$D = (4, 4), \quad E = (5, 1), \quad F = (2, 2).$$

The pivot is

$$A = (0, 0)$$

because it has the smallest y -coordinate.



10 Polar-Angle Order

Around pivot A , the points have the following angle order:

$$\tan \theta = \frac{y - y_A}{x - x_A}.$$

Since $A = (0, 0)$, this becomes

$$\tan \theta = \frac{y}{x}, \quad \theta = \tan^{-1}\left(\frac{y}{x}\right).$$

Point	Vector from A	Polar angle
$E = (5, 1)$	$(5, 1)$	11.31°
$B = (2, 1)$	$(2, 1)$	26.565°
$F = (2, 2)$	$(2, 2)$	45°
$D = (4, 4)$	$(4, 4)$	45°
$C = (1, 3)$	$(1, 3)$	71.565°

Initial order:

$$A, E, B, F, D, C.$$

Since F and D have the same polar angle, compare distances:

$$AF^2 = 2^2 + 2^2 = 8, \quad AD^2 = 4^2 + 4^2 = 32.$$

Since

$$32 > 8,$$

keep D and discard F .

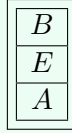
Effective Scan Order

$$A, E, B, D, C.$$

11 Scan Trace

Stack Notation

In this trace, the stack is drawn vertically with the top item in the highest cell:



Here A is at the bottom, B is at the top, and the next orientation test uses the top two points.

11.1 Start with A, E, B

Push:

A, E, B .

So the stack becomes



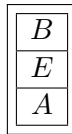
Check:

$\text{orient}(A, E, B)$.

$$\overrightarrow{AE} = (5, 1), \quad \overrightarrow{AB} = (2, 1).$$

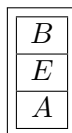
$$\text{orient}(A, E, B) = 5(1) - 1(2) = 5 - 2 = 3.$$

Since $3 > 0$, this is a left turn. Keep B . The stack remains



11.2 Process D

The next point is D . Before inserting D , the stack is



Check:

$\text{orient}(E, B, D)$.

$$\overrightarrow{EB} = (2 - 5, 1 - 1) = (-3, 0),$$

$$\overrightarrow{ED} = (4 - 5, 4 - 1) = (-1, 3).$$

$$\text{orient}(E, B, D) = (-3)(3) - 0(-1) = -9.$$

Since $-9 < 0$, this is a right turn. Pop B .

After popping B , the stack becomes



Now check:

$$\text{orient}(A, E, D).$$

$$\text{orient}(A, E, D) = 5(4) - 1(4) = 20 - 4 = 16.$$

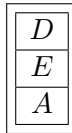
Since $16 > 0$, push D .

After pushing D , the stack becomes



11.3 Process C

The next point is C . Before inserting C , the stack is



Check:

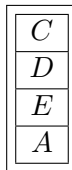
$$\text{orient}(E, D, C).$$

$$\overrightarrow{ED} = (-1, 3), \quad \overrightarrow{EC} = (1 - 5, 3 - 1) = (-4, 2).$$

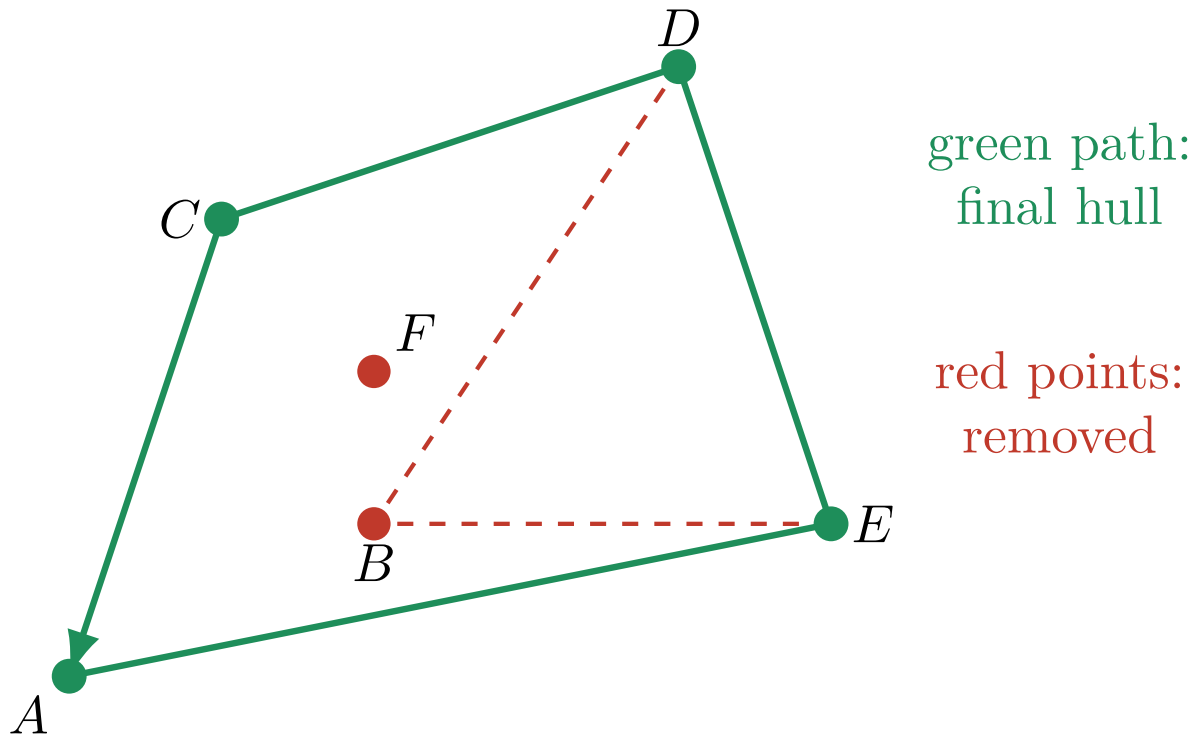
$$\text{orient}(E, D, C) = (-1)(2) - 3(-4) = -2 + 12 = 10.$$

Since $10 > 0$, push C .

After pushing C , the final stack is



12 Visual Trace



13 Final Hull

The final stack is:

top $\rightarrow C, D, E, A.$

So the convex hull in boundary order is:

$A \rightarrow E \rightarrow D \rightarrow C \rightarrow A.$

The removed points are:

B and $F.$

14 Second Worked Example

Problem

Apply Graham Scan to

$$P_0 = (0, 0), \quad P_1 = (1, 1), \quad P_2 = (2, 0), \quad P_3 = (2, 2), \quad P_4 = (0, 2).$$

The pivot is already

$$P_0 = (0, 0),$$

because it has the smallest y -coordinate and then the smallest x -coordinate.

Sorted Order

Since $P_0 = (0, 0)$, each polar angle can be found from

$$\tan \theta = \frac{y}{x}.$$

Point	Angle calculation	Polar angle
$P_2 = (2, 0)$	$\tan \theta = \frac{0}{2} = 0$	0°
$P_1 = (1, 1)$	$\tan \theta = \frac{1}{1} = 1$	45°
$P_3 = (2, 2)$	$\tan \theta = \frac{2}{2} = 1$	45°
$P_4 = (0, 2)$	vertical ray from P_0	90°

Therefore the initial polar-angle order around P_0 is

$$\boxed{P_2, P_1, P_3, P_4.}$$

Points P_1 and P_3 have the same polar angle 45° . Compare squared distances:

$$d^2(P_0, P_1) = 1^2 + 1^2 = 2,$$

$$d^2(P_0, P_3) = 2^2 + 2^2 = 8.$$

Since P_3 is farther, discard P_1 . The point P_1 lies on the line segment from P_0 to P_3 , so it is not a corner vertex.

Effective scan order:

$$\boxed{P_0, P_2, P_3, P_4.}$$

Scan

Start with

$$P_0, P_2, P_3.$$

The stack becomes

$$\boxed{\begin{array}{c} P_3 \\ P_2 \\ P_0 \end{array}}$$

Check:

$$\text{orient}(P_0, P_2, P_3).$$

$$\overrightarrow{P_0P_2} = (2, 0), \quad \overrightarrow{P_0P_3} = (2, 2).$$

$$\text{orient}(P_0, P_2, P_3) = 2(2) - 0(2) = 4 > 0.$$

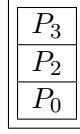
So this is a left turn. Keep P_3 , and the stack stays

$$\boxed{\begin{array}{c} P_3 \\ P_2 \\ P_0 \end{array}}$$

Now process P_4 :

$$\text{orient}(P_2, P_3, P_4).$$

Before inserting P_4 , the stack is

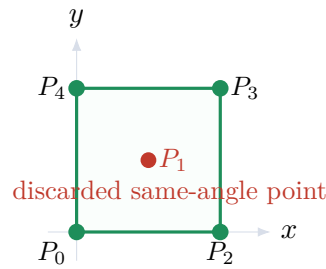
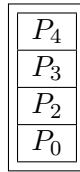


$$\overrightarrow{P_2P_3} = (0, 2), \quad \overrightarrow{P_2P_4} = (-2, 2).$$

$$\text{orient}(P_2, P_3, P_4) = 0(2) - 2(-2) = 4 > 0.$$

So this is also a left turn. Push P_4 .

The final stack is



Second Example Result

$$P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0$$

is the convex hull. Point P_1 is discarded because it is nearer than P_3 on the same ray from the pivot.

15 Complexity

Choosing the pivot takes $\mathcal{O}(n)$. Sorting the remaining points by polar angle takes

$$\mathcal{O}(n \log n).$$

The stack scan takes $\mathcal{O}(n)$, because each point is pushed once and popped at most once.

Time Complexity

$$\mathcal{O}(n) + \mathcal{O}(n \log n) + \mathcal{O}(n) = \mathcal{O}(n \log n).$$

16 Final Review

Graham Scan Summary

Choose pivot P_0 with minimum y , then minimum x .
Sort all other points by polar angle around P_0 .
For equal angles, keep the farthest point.
Use a stack and pop while orientation ≤ 0 .
Running time is $\mathcal{O}(n \log n)$.